

گزارش پروژه

طراحی و پیاده‌سازی سامانه پایش وضعیت راننده با استفاده از Vision Transformer برای

تشخیص رفتارهای خطرناک در تصاویر داخل خودرو

مقدمه و هدف پروژه

هدف اصلی در این پروژه هدف، طراحی یک سامانه‌ی پایش وضعیت راننده با استفاده از مدل‌های بینایی مبتنی بر Vision Transformer (ViT) است؛ سامانه‌ای که بتواند با تحلیل تصاویر داخل خودرو، رفتارها و وضعیت‌های خطرناک راننده را تشخیص دهد و به افزایش ایمنی رانندگی کمک کند. این وضعیت‌ها می‌توانند شامل حواس‌پرتی راننده (مثل استفاده از موبایل، نگاه کردن به اطراف، صحبت با سرنشین و ...)، خواب‌آلودگی/چرت‌زدن، یا سایر حالت‌هایی باشند که احتمال وقوع حادثه را بالا می‌برند. برای رسیدن به این هدف، ابتدا یک مجموعه‌داده‌ی مناسب از تصاویر راننده انتخاب و آماده‌سازی می‌شود. سپس با توجه به داده‌ی انتخاب‌شده، مراحل پیش‌پردازش انجام می‌گیرد و کلاس‌های خروجی مطابق هدف مسئله تعیین می‌شوند. در ادامه، ساختار Vision Transformer بررسی شده و نحوه‌ی استفاده از آن برای طبقه‌بندی تصاویر توضیح داده می‌شود.

بخش اصلی پروژه مربوط به پیاده‌سازی و آموزش مدل است: یک مدل ViT به صورت پیاده‌سازی سفارشی و از ابتدا (بدون استفاده از مدل‌های از پیش تعریف‌شده) ساخته می‌شود و فرآیند آموزش آن انجام می‌گیرد. همچنین برای آموزش، حداقل دو بهینه‌ساز مختلف با هم مقایسه می‌شوند و پارامترهای آموزشی (مانند نرخ یادگیری، وزن‌دهی منظم‌سازی، زمان‌بندی یادگیری و ...) با هدف رسیدن به عملکرد بهتر و اجرای بهینه تنظیم می‌گردند. در پایان، نتایج مدل با معیارهای مناسب گزارش می‌شود و تحلیل خطا انجام می‌گیرد تا مشخص شود مدل در چه مواردی دچار اشتباه می‌شود و چرا.

۱- انتخاب و آماده‌سازی مجموعه‌داده

در این پروژه برای پیاده‌سازی سامانه پایش وضعیت راننده، مجموعه‌داده State Farm Distracted Driver Detection انتخاب و آماده‌سازی شد. این مجموعه‌داده شامل تصاویر واقعی از داخل خودرو و برچسب‌های مربوط به فعالیت‌های مختلف راننده است و برای مسئله تشخیص حواس‌پرتی راننده مناسب می‌باشد. یکی از مزیت‌های مهم این دیتاست، وجود شناسه راننده (subject) در فایل برچسب‌ها است که امکان تقسیم‌بندی اصولی داده‌ها و جلوگیری از نشت اطلاعات بین آموزش و آزمون را فراهم می‌کند.

مراحل انجام شده در کد برای آماده سازی داده ها به صورت زیر بود:

۱. دریافت داده ها از Kaggle با استفاده از Kaggle API و دانلود فایل فشرده مسابقه state-farm-distracted-driver-detection.

۲. استخراج فایل های فشرده در مسیر مشخص و ایجاد یک فایل نشانه برای جلوگیری از استخراج تکراری در اجرای مجدد نوت بوک.

۳. یافتن فایل driver_imgs_list.csv و پوشه تصاویر آموزشی، سپس خواندن فایل CSV که برای هر تصویر، کلاس و شناسه راننده را مشخص می کند.

۴. ساخت مسیر کامل هر تصویر بر اساس ساختار پوشه ها (train/classname/image) و بررسی وجود فایل ها برای جلوگیری از خطاهای مسیر.

۵. استخراج نام کلاس ها و ساخت نگاشت کلاس به شناسه عددی برای استفاده در مرحله آموزش مدل.

خلاصه آماری داده پس از آماده سازی مطابق خروجی نوت بوک:

- تعداد کل نمونه ها: ۲۲۴۲۴ تصویر

- تعداد راننده ها: ۲۶

- تعداد کلاس ها: ۱۰ کلاس با نام های c۰ تا c۹

پس از آماده سازی اولیه، تقسیم بندی داده ها به صورت driver-based انجام شد تا هیچ راننده ای همزمان در مجموعه آموزش و آزمون قرار نگیرد. مطابق خروجی های ثبت شده، اندازه مجموعه ها به شکل Train=۱۵۵۳۸، Val=۳۳۶۱ و Test=۳۵۲۵ بوده و بررسی همپوشانی نشان داد Driver overlap train/test برابر ۰ است. این موضوع باعث می شود ارزیابی مدل روی راننده های دیده نشده انجام شود و نتایج به شرایط واقعی نزدیک تر باشد. در پایان این مرحله، داده ها به شکل یک DataFrame استاندارد شامل مسیر تصویر، کلاس و شناسه راننده آماده شد تا در مراحل بعدی برای تقسیم بندی نهایی، ساخت Dataset و DataLoader و آموزش مدل استفاده شود.

۲- پیش پردازش داده‌ها، تعیین کلاس‌های خروجی و ویژگی‌های مورد نیاز سامانه

در این مرحله، با توجه به مجموعه داده انتخاب شده، فرآیند پیش‌پردازش تصاویر و تعریف خروجی‌های مدل انجام شد تا داده‌ها برای آموزش یک مدل Vision Transformer مناسب شوند و هدف سامانه به شکل دقیق مشخص گردد. کلاس‌های خروجی در دیتاست State Farm Distracted Driver Detection شامل ۱۰ حالت مختلف رانندگی با برچسب‌های C۰ تا C۹ است. این کلاس‌ها نشان‌دهنده فعالیت‌ها و وضعیت‌های متفاوت راننده هستند و مدل باید بتواند با دریافت تصویر، یکی از این ۱۰ کلاس را به عنوان خروجی تشخیص دهد. در کد پروژه، برای استفاده در آموزش، یک نگاشت از نام کلاس‌ها به شناسه عددی ساخته شد و برچسب هر تصویر به صورت عددی به مدل داده شد.

برای پیش‌پردازش، تصاویر ابتدا به اندازه ثابت برای ورودی مدل تبدیل شدند و سپس تبدیلات لازم برای آماده‌سازی ورودی ViT اعمال گردید. این عملیات شامل تغییر اندازه تصویر به ابعاد مشخص، تبدیل تصویر به Tensor و نرمال‌سازی کانال‌ها بود تا توزیع داده‌ها پایدارتر شود و آموزش بهتر انجام گیرد. همچنین برای افزایش تنوع داده و کاهش بیش‌برازش، روی تصاویر آموزشی از تبدیلات افزایشی استفاده شد. این افزایشی‌سازی شامل عملیات‌هایی مانند برش تصادفی، وارونگی افقی و تغییرات جزئی در رنگ و روشنایی بود تا مدل نسبت به تغییرات طبیعی در شرایط واقعی مقاوم‌تر شود. برای داده‌های اعتبارسنجی و آزمون، فقط تبدیلات پایه و قطعی اعمال شد تا ارزیابی منصفانه و قابل مقایسه باشد. در کنار پیش‌پردازش، برای کاهش اثر عدم توازن کلاس‌ها در آموزش از نمونه‌برداری وزن‌دار استفاده شد تا احتمال انتخاب کلاس‌های کم‌نمونه در هر batch افزایش یابد و مدل فرصت یادگیری بهتری روی همه کلاس‌ها داشته باشد.

در نهایت، ویژگی‌های مورد نیاز برای سامانه پایش وضعیت راننده بر اساس این مسئله شامل موارد زیر در نظر گرفته شد:

۱. توانایی تشخیص فعالیت‌های حواس‌پرتی راننده در چند کلاس مشخص
۲. دقت قابل قبول و پایداری عملکرد روی راننده‌های دیده‌نشده
۳. سرعت مناسب در مرحله استنتاج برای استفاده در سناریوهای نزدیک به زمان واقعی
۴. مقاومت نسبت به تغییرات نور، زاویه دوربین، پس‌زمینه و تفاوت افراد

خروجی این مرحله، ساخت Dataset و DataLoader برای آموزش و ارزیابی بود که تصاویر را با پیش پردازش مناسب و برچسب عددی به مدل ارائه می دهد.

۳- بررسی ساختار Vision Transformer و نحوه استفاده از آن در این مسئله

Vision Transformer یک معماری مبتنی بر ترنسفورمر است که به جای پردازش ترتیبی متن، تصویر را به شکل یک دنباله از توکن ها وارد ترنسفورمر می کند. ایده اصلی این است که تصویر به patch های کوچک تر تقسیم شود و هر patch مانند یک توکن در مدل های زبانی پردازش گردد. در این پروژه نیز همین ساختار برای طبقه بندی تصاویر راننده به ۱۰ کلاس استفاده شد.

جریان کلی پردازش در ViT به صورت زیر است:

۱. تبدیل تصویر به patch ها تصویر ورودی با اندازه ثابت به patch های هم اندازه تقسیم می شود. اگر اندازه patch برابر ۱۶ باشد، تصویر 224×224 به شبکه ای 14×14 تبدیل می شود و در نتیجه ۱۹۶ patch تولید می گردد. هر patch سپس به یک بردار تبدیل شده و flatten می شود.

۲. Patch Embedding: هر patch پس از flatten شدن، با یک لایه خطی به فضای ویژگی با بعد ثابت نگاشت می شود. این مرحله باعث می شود تمام patch ها به بردارهایی با ابعاد یکسان (embedding dimension) تبدیل شوند تا بتوانند وارد ترنسفورمر شوند.

۳. اضافه کردن class token و positional embedding: برای انجام طبقه بندی، یک بردار یادگرفتنی به نام class token به ابتدای دنباله توکن ها اضافه می شود. همچنین برای اینکه مدل اطلاعات مکانی patch ها را از دست ندهد، یک positional embedding یادگرفتنی به همه توکن ها افزوده می شود. در پایان این مرحله، مدل یک دنباله شامل (تعداد patch ها + ۱) توکن را دریافت می کند.

۴. عبور از Encoder Blocks: دنباله توکن ها از چندین بلوک ترنسفورمری عبور داده می شود. هر Encoder Block معمولاً شامل این بخش هاست:

- LayerNorm
- Multi-Head Self Attention برای مدل کردن ارتباط بین patch ها و استخراج وابستگی های سراسری تصویر

- اتصال میان بر (Residual Connection) برای پایداری آموزش

- MLP یا Feed Forward Network برای یادگیری تبدیل‌های غیرخطی و غنی‌تر کردن نمایش ویژگی‌ها

- Dropout برای کاهش بیش‌برازش

در پیاده‌سازی این پروژه، این ساختار به صورت ماژول‌های مجزا ساخته شد و سپس در قالب یک کلاس VisionTransformer کنار هم قرار گرفت.

۵. Head طبقه‌بندی: پس از عبور از بلوک‌های ترنسفورمری، بردار مربوط به class token به عنوان خلاصه‌ای از کل تصویر در نظر گرفته می‌شود. این بردار وارد یک لایه خطی (classifier head) می‌شود تا logits مربوط به ۱۰ کلاس تولید گردد. سپس با softmax می‌توان احتمال هر کلاس را به دست آورد.

نحوه استفاده از این ساختار برای مسئله پایش وضعیت راننده در پروژه:

در مسئله حاضر، هدف تشخیص وضعیت راننده از روی تصویر داخل خودرو است. مدل ViT به دلیل استفاده از self-attention می‌تواند به صورت همزمان ارتباط بین بخش‌های مختلف تصویر را یاد بگیرد، مثلاً ارتباط بین دست‌ها، صورت، گوشی، فرمان و جهت نگاه. این ویژگی برای تشخیص کلاس‌هایی که تفاوت آن‌ها به تعامل چند ناحیه تصویر وابسته است بسیار مفید است. در این پروژه، ViT سفارشی به گونه‌ای طراحی شد که با ورودی تصاویر پایش‌پردازش‌شده، در نهایت یکی از کلاس‌های ۲۰ تا ۲۹ را پیش‌بینی کند. خروجی نهایی مدل به صورت logits ده‌بعدی تولید شده و با استفاده از تابع هزینه Cross Entropy برای یادگیری استفاده شد.

۴- پیاده‌سازی Vision Transformer از ابتدا، آموزش مدل و مقایسه دو بهینه‌ساز

در این بخش، یک مدل Vision Transformer به صورت سفارشی و از ابتدا پیاده‌سازی شد و سپس مدل از صفر آموزش داده شد. همچنین برای آموزش، دو بهینه‌ساز AdamW و SGD تحت یک تنظیم آموزشی ثابت اجرا شدند تا عملکرد آن‌ها مقایسه و بهترین گزینه انتخاب شود.

۴-۱- پیاده‌سازی معماری ViT از ابتدا

در کد پروژه، اجزای اصلی ViT به صورت کلاس‌های جدا پیاده‌سازی و سپس در کلاس VisionTransformer جمع شدند. این اجزا شامل تبدیل تصویر به توکن‌های patch (Patch Embedding)، افزودن class

Encoder (LayerNorm + Multi-Head Self positional embedding، token، چند بلاک، Attention + MLP + residual)، و در نهایت head طبقه‌بندی برای تولید logits کلاس‌ها بود .

پیکربندی مدل (MODEL_CFG) :

Value	Parameter	Value	Parameter
۶	heads	۲۲۴	img_size
۴	mlp_ratio	۱۶	patch_size
۰.۱	drop	۳۸۴	embed_dim
۰	attn_drop	۶	depth

این پیکربندی باعث می‌شود تعداد patch ها برای ورودی ۲۲۴ با patch_size=۱۶ برابر ۱۹۶ باشد که هزینه محاسباتی self-attention را کنترل می‌کند. همچنین depth=۶ نسبت به مدل‌های بزرگ‌تر سبک‌تر است.

۲-۴- تنظیمات آموزش و پایپ‌لاین آموزشی

برای داشتن آموزش پایدار و قابل تکرار، موارد زیر در کد اعمال شد.

Setting	Training Item
۳۵	epochs
۱۰	early stopping patience
label_smoothing=۰.۱ با CrossEntropy	loss
warmup + cosine (warmup = ۱۰٪ گام‌ها)	scheduler
۶۴	batch size
autocast + GradScaler	mixed precision
۱.۰	grad clipping
WeightedRandomSampler برای کاهش اثر عدم توازن کلاس‌ها	sampler

در مرحله آموزش، از data augmentation شامل RandomResizedCrop، RandomHorizontalFlip و ColorJitter استفاده شد و برای val/test فقط تبدیلات ثابت اعمال گردید.

همچنین برای کاهش اثر عدم توازن کلاس‌ها از **WeightedRandomSampler** استفاده شد و تقسیم‌بندی داده‌ها به صورت **driver-based** انجام شد تا از نشت اطلاعات جلوگیری شود.

۴-۳- نتایج آموزش و مقایسه دو بهینه‌ساز

در این مرحله، دو اجرای مستقل با معماری و تنظیمات یکسان انجام شد و تنها بهینه‌ساز تغییر کرد. معیار اصلی انتخاب، بهترین دقت اعتبارسنجی بود. مطابق خروجی‌های به‌دست‌آمده، **AdamW** بهترین دقت اعتبارسنجی را در **epoch ۱۹** ثبت کرد و **SGD** بهترین دقت اعتبارسنجی را در **epoch ۲۷** به دست آورد.

نتایج مقایسه:

Early Stop	Best Epoch	Best Val Acc	Nesterov	Momentum	Weight Decay	LR	Optimizer
۲۹	۱۹	۰.۴۵۷۳	-	-	۰.۰۵	۰.۰۰۰۳	AdamW
-	۲۷	۰.۳۴۸۴	True	۰.۹	۰.۰۰۰۱	۰.۰۳	SGD

۴-۴- تحلیل و توضیح نتایج

الف) انتخاب بهینه‌ساز بهتر :

AdamW با بهترین دقت اعتبارسنجی **۰.۴۵۷۳** عملکرد بهتری نسبت به **SGD** با بهترین دقت **۰.۳۴۸۴** داشت. بنابراین طبق معیار اعتبارسنجی، **AdamW** به عنوان گزینه نهایی انتخاب شد.

ب) رفتار همگرایی و پایداری آموزش :

در اجرای **AdamW** روند یادگیری سریع‌تر و پایداری بود و دقت اعتبارسنجی در چند نقطه به محدوده بالای **۰.۴۳** رسید و در **epoch ۱۹** به بهترین مقدار خود (**۰.۴۵۷۳**) رسید. در مقابل، **SGD** با وجود اینکه تا پایان **epoch ۳۵** ادامه پیدا کرد، بهبود کندتری داشت و بهترین مقدار آن پایین‌تر از **AdamW** باقی ماند. این رفتار نشان می‌دهد در این مسئله و با این تنظیمات، **AdamW** برای آموزش **ViT** از صفر انتخاب مناسب‌تری است.

ج) نکات مربوط به بهینه بودن اجرا و مصرف منابع:

برای بهینه‌تر شدن آموزش و کاهش هزینه محاسباتی، چند تصمیم کلیدی در کد اعمال شد: استفاده از **patch_size=۱۶** برای محدود کردن تعداد توکن‌ها، انتخاب **depth=۶** برای سبک‌تر بودن مدل، استفاده از

mixed precision برای کاهش مصرف حافظه و افزایش سرعت روی GPU ، و Early Stopping برای جلوگیری از آموزش غیرضروری، و به کارگیری warmup + cosine برای پایداری بیشتر در ابتدای آموزش.

جمع بندی

در این بخش، مدل Vision Transformer به صورت کامل از ابتدا پیاده سازی و از صفر آموزش داده شد. سپس AdamW و SGD با شرایط برابر مقایسه شدند و AdamW با بهترین دقت اعتبارسنجی ۰.۴۵۷۳ نسبت به SGD با ۰.۳۴۸۴ برتری داشت و به عنوان بهینه ساز نهایی انتخاب شد.

۵- گزارش نتایج پیش بینی مدل با معیارهای مناسب

در این بخش، پس از انتخاب بهینه ساز AdamW به عنوان بهینه ساز برتر ، مدل نهایی روی مجموعه آزمون ارزیابی شد تا عملکرد آن روی داده های دیده نشده سنجیده شود. ارزیابی با استفاده از معیارهای استاندارد انجام شد و علاوه بر گزارش معیارهای کلی، ماتریس آشفتگی نیز برای بررسی الگوی خطاهای مدل تولید گردید. کارهای انجام شده در پروژه:

۵-۱- اجرای استنتاج روی مجموعه آزمون

مدل نهایی روی test loader اجرا شد و برای هر تصویر آزمون، خروجی logits محاسبه شد. سپس با انتخاب بیشترین مقدار logits ، کلاس نهایی پیش بینی شده به دست آمد. همزمان برچسب واقعی و برچسب پیش بینی شده برای محاسبه معیارها ذخیره شد.

۵-۲- محاسبه معیارهای ارزیابی

برای گزارش عملکرد کلی مدل از دو معیار اصلی استفاده شد:

- Accuracy: برای سنجش درصد پیش بینی های درست روی کل نمونه ها
- Macro F۱: برای ارزیابی متوازن تر بین کلاس ها و کاهش اثر عدم توازن داده ها

۵-۳- تولید ماتریس آشفتگی

ماتریس آشفتگی برای نمایش دقیق خطاهای بین کلاس ها تولید و رسم شد تا مشخص شود مدل کدام کلاس ها را بیشتر با یکدیگر اشتباه می گیرد.

نتایج و خروجی‌های بدست آمده :

Value	Metric
AdamW	بهینه‌ساز منتخب
۰.۴۵۷۳	بهترین دقت اعتبارسنجی
۱۹	بهترین epoch
۳۵۲۵	تعداد نمونه‌های آزمون
۰.۴۴۳۷	Test Accuracy
۰.۴۴۰۱	Test Macro F۱
۱۰×۱۰	ابعاد ماتریس آشفتگی

۵-۴- تحلیل نتایج

۱. اختلاف بین test و validation :

بهترین دقت اعتبارسنجی ۰.۴۵۷۳ و دقت آزمون ۰.۴۴۳۷ است (اختلاف حدود ۱.۴ درصد). این مقدار اختلاف معمولاً طبیعی است چون test کاملاً دیده‌نشده است و معمولاً عملکرد روی آن کمی پایین‌تر از validation می‌شود. کم بودن این اختلاف نشان می‌دهد افت عملکرد شدید نیست و تعمیم‌دهی مدل نسبتاً پایدار است. همچنین ارزیابی آزمون با همان مدلی انجام شده که در طول آموزش بهترین دقت validation را ثبت کرده و ذخیره شده بود. (۱۹ epoch)

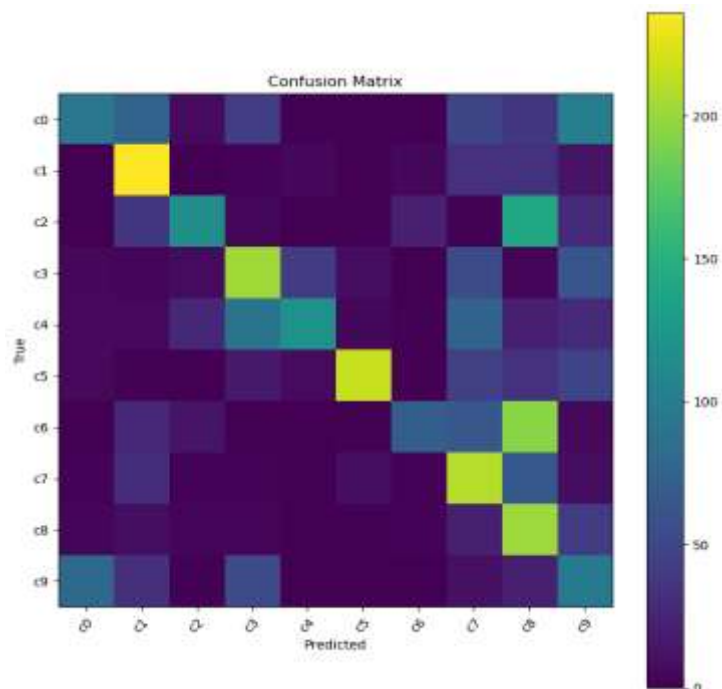
۲. اهمیت Macro F۱ در این مسئله:

Macro F۱ برابر ۰.۴۴۰۱ نزدیک به Accuracy است. این موضوع نشان می‌دهد عملکرد مدل روی کلاس‌ها به طور کامل به یک یا چند کلاس پرتکرار محدود نشده و مدل در مجموع رفتار نسبتاً متوازنی بین کلاس‌ها داشته است، هرچند همچنان در تفکیک برخی کلاس‌های مشابه دچار خطا می‌شود.

۵-۵- تحلیل ماتریس آشفتگی

در این ماتریس، محور عمودی کلاس واقعی و محور افقی کلاس پیش‌بینی شده را نشان می‌دهد. هر خانه تعداد نمونه‌هایی را نمایش می‌دهد که از یک کلاس واقعی به یک کلاس

پیش‌بینی‌شده افتاده‌اند. رنگ‌های روشن‌تر (زرد/سبز روشن) یعنی تعداد بیشتر و رنگ‌های تیره‌تر (بنفش) یعنی تعداد کمتر.



الگوی کلی رنگ‌ها:

- خانه‌های روی قطر اصلی اگر روشن باشند یعنی مدل آن کلاس را درست تشخیص داده است.
 - خانه‌های روشن خارج از قطر اصلی یعنی خطای پرتکرار بین دو کلاس مشخص.
- کلاس‌هایی که روی قطر اصلی روشن‌تر هستند:
- c1، c3، c5، c7 و c8 روی قطر اصلی رنگ نسبتاً روشن‌تری دارند؛ یعنی مدل در این کلاس‌ها تعداد بیشتری پیش‌بینی درست داشته است.
 - c0، c6 و c9 روی قطر اصلی به نسبت تیره‌تر هستند؛ یعنی مدل در این کلاس‌ها نمونه‌های درست کمتری داشته و بیشتر دچار اشتباه شده است.

خطاهای پرتکرار (خانه‌های روشن خارج از قطر اصلی):

- در ردیف C۶ (کلاس واقعی C۶) ، یک خانه بسیار روشن در ستون C۸ دیده می‌شود. این یعنی بخش بزرگی از نمونه‌های C۶ به اشتباه C۸ پیش‌بینی شده‌اند.
- در ردیف C۲ نیز خانه نسبتاً روشن در ستون C۸ دیده می‌شود، یعنی C۲ هم زیاد با C۸ اشتباه شده است.
- بین C۰ و C۹ دوطرفه بودن اشتباه قابل مشاهده است. یعنی هم C۰ گاهی به C۹ می‌افتد و هم C۹ گاهی به C۰. این حالت معمولاً وقتی رخ می‌دهد که تفاوت رفتاری در تصویر خیلی ظریف باشد و نشانه‌های تمایزبخش واضح نباشد.
- بین C۴ و C۳ نیز یک الگوی اشتباه قابل توجه دیده می‌شود که نشان‌دهنده شباهت بصری این دو کلاس است.
- برای C۷ نیز مقدار قابل توجهی به سمت C۸ دیده می‌شود، یعنی C۷ و C۸ هم در برخی تصاویر به هم نزدیک هستند و مدل بین آن‌ها جابه‌جایی دارد.

جمع‌بندی تفسیری از روی رنگ‌ها

این ماتریس نشان می‌دهد بیشترین مشکل مدل مربوط به چند مرز مشخص بین کلاس‌هاست، به‌خصوص تمایل مدل به پیش‌بینی C۸ به جای برخی کلاس‌ها مثل C۶ و C۲۰. همچنین چند جفت کلاس دیگر مثل C۰ و C۹ و نیز C۳ و C۴ هم شباهت دارند و باعث خطای دوطرفه یا یک‌طرفه می‌شوند. به همین دلیل در بخش تحلیل خطا (سوال ۶) تمرکز روی همین جفت‌های پرتکرار منطقی است، چون بخش قابل توجهی از خطاهای کلی از همین نواحی روشن خارج از قطر اصلی می‌آید.

۶- تحلیل خطاهای مدل و بررسی موارد ناتوانی در پیش‌بینی صحیح

در این بخش، خروجی‌های مرحله آزمون برای بررسی خطاهای مدل تحلیل شد. ابتدا یک جدول از نتایج پیش‌بینی ساخته شد (مسیر تصویر، کلاس واقعی، کلاس پیش‌بینی‌شده) و سپس نمونه‌های اشتباه جدا شدند تا الگوی خطاها مشخص شود.

خلاصه وضعیت خطاها:

طبق خروجی پروژه، تعداد کل نمونه‌های آزمون ۳۵۲۵ است و تعداد پیش‌بینی‌های اشتباه ۱۹۶۱ گزارش شده

است. این یعنی حدود ۵۵.۶ درصد نمونه‌ها اشتباه پیش‌بینی شده‌اند. این مقدار خطا نشان می‌دهد مسئله دارای کلاس‌های با شباهت بصری بالا است و مدل در تشخیص نشانه‌های ظریف (مثل وضعیت دست، حضور موبایل/شیء، جهت نگاه) با چالش مواجه می‌شود.

پرتکرارترین جفت‌های اشتباه (Top Confusion Pairs):

با گروه‌بندی اشتباهات به صورت (کلاس واقعی، کلاس پیش‌بینی شده)، پرتکرارترین خطاها استخراج شده‌اند. جدول زیر خلاصه چند مورد اول را نشان می‌دهد.

جدول جفت‌های پرتکرار خطا

کلاس واقعی	کلاس پیش‌بینی	تعداد	کلاس واقعی	کلاس پیش‌بینی	تعداد
C۶	C۸	۱۹۴	C۴	C۷	۷۵
C۲	C۸	۱۴۱	C۰	C۱	۷۵
C۰	C۹	۱۰۱	C۷	C۸	۶۵
C۴	C۳	۹۱	C۶	C۷	۶۴
C۹	C۰	۸۱	C۳	C۹	۶۲

تفسیر کلی این جدول:

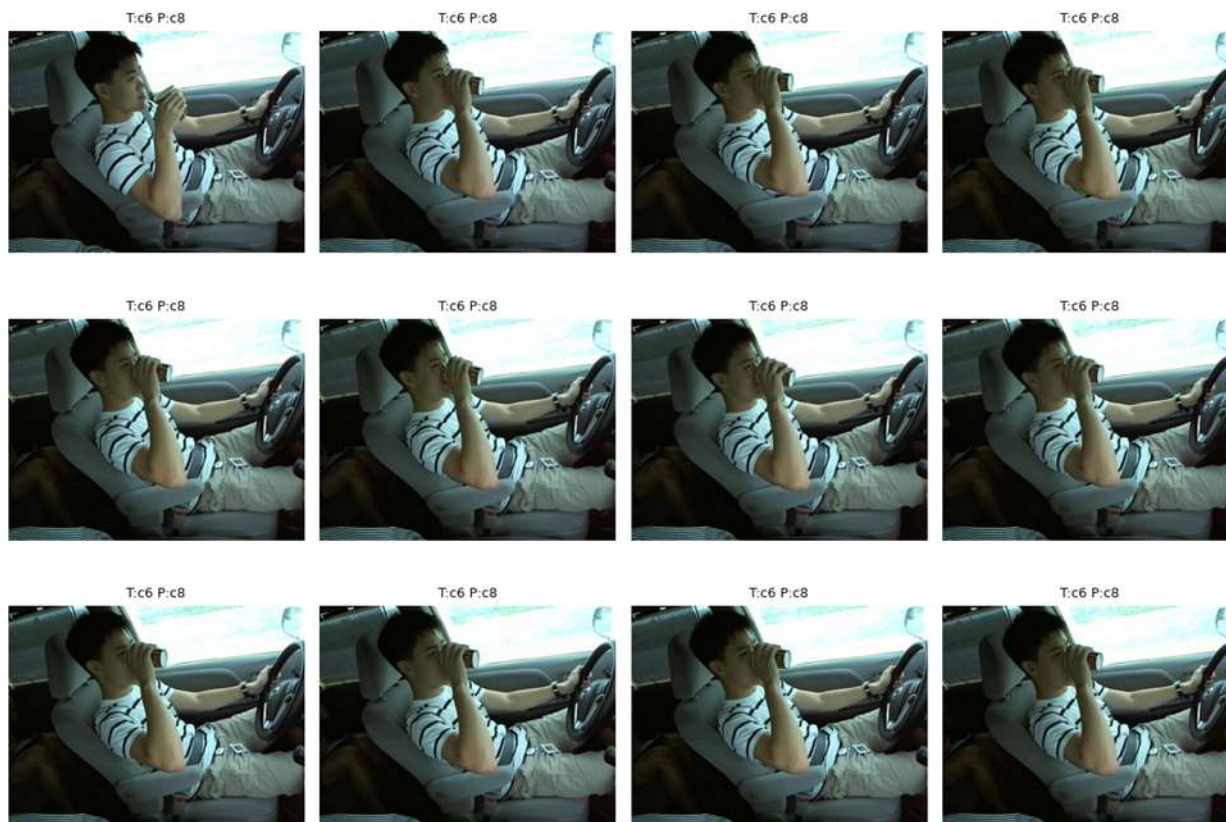
الف) تمرکز چند خطای بزرگ روی چند جفت خاص نشان می‌دهد مدل در مرز بین برخی رفتارهای مشابه گیر می‌کند.

ب) وجود خطاهای دوطرفه مثل $C۰ \rightarrow C۹$ و $C۹ \rightarrow C۰$ نشان می‌دهد دو کلاس از نظر بصری در بسیاری از تصاویر به هم نزدیک هستند و تغییرات کوچک (مثلاً زاویه سر یا نگاه) تعیین‌کننده می‌شود.

تحلیل کیفی با مثال تصویری:

الف) پرتکرارترین خطا $C۶ \rightarrow C۸$: در خروجی سلول ۲۴ چند نمونه از این خطا نمایش داده شده است. در تصاویر دیده می‌شود که دست راننده نزدیک صورت قرار دارد و یک شیء (مثل بطری/لیوان) در ناحیه صورت دیده می‌شود؛ این حالت از نظر الگوی ظاهری می‌تواند با حالتی که دست برای کار نزدیک صورت/سر استفاده می‌شود مشابه شود و مدل نتواند تفاوت را با اطمینان تشخیص دهد.

Most frequent confusion: c6 -> c8 | count=194



علت‌های محتمل این خطا:

- شباهت ژست دست: در هر دو کلاس، دست در ناحیه صورت یا نزدیک آن قرار می‌گیرد.
- کوچک بودن نشانه‌های تمایزبخش: شیء داخل دست (بطری/گوشی/لوازم) ممکن است کوچک، تار یا در نور زیاد/کم محو شده باشد.
- تاثیر نور و بازتاب شیشه: روشنایی شدید پنجره و پس‌زمینه می‌تواند جزئیات دست و شیء را کم‌رنگ کند.

(ب) نمونه‌های تصادفی از خطاها:

خروجی سلول ۲۵ چند نمونه اشتباه را به صورت تصادفی نشان می‌دهد و کمک می‌کند مشخص شود خطاهای مدل فقط محدود به یک جفت کلاس خاص نیست. این تصاویر معمولاً نشان می‌دهند بخشی از اشتباهات به

شرایط تصویر وابسته است؛ مثل زاویه دوربین، تغییر نور و سایه داخل خودرو، پوشانده شدن بخشی از دست یا صورت، یا حضور سرنشین و اشیای اضافی که باعث می‌شود نشانه‌های لازم برای تشخیص کلاس واضح نباشد.



۸- افزایش سرعت مرحله استنتاج و مقایسه با حالت قبل

در این بخش، هدف این بود که سرعت مرحله استنتاج مدل افزایش پیدا کند و همزمان بررسی شود که بهینه‌سازی انجام‌شده باعث تغییر معنی‌دار در خروجی مدل نشده باشد. برای این کار، سه سلول پایانی (۲۷، ۲۸ و ۲۹) به ترتیب برای آماده‌سازی بنچمارک، اجرای بنچمارک و مقایسه صحت خروجی استفاده شدند.

سلول ۲۷: آماده‌سازی DataLoader مخصوص بنچمارک (bench_loader) در این سلول، یک DataLoader جداگانه روی مجموعه آزمون ساخته شد که ویژگی مهم آن drop_last=True است. این گزینه باعث می‌شود همه batch ها اندازه ثابت داشته باشند و آخرین batch ناقص حذف شود. ثابت بودن اندازه batch برای بنچمارک سرعت اهمیت دارد، چون تغییر شکل ورودی می‌تواند باعث نوسان در زمان اجرا

و غیرقابل مقایسه شدن نتایج شود. همچنین مقدار shuffle=False قرار داده شد تا ترتیب داده‌ها ثابت باشد و num_workers و pin_memory نیز برای بهبود کارایی بارگذاری داده‌ها تنظیم شدند. خروجی نشان داد bench_loader شامل ۵۵ batch با batch_size=۶۴ است و drop_last فعال است.

خلاصه تنظیمات bench_loader :

مورد	مجموعه داده	batch size	تعداد batch	shuffle	drop_last	num_workers	pin_memory
مقدار	test_ds	۶۴	۵۵	False	True	۴	True (CUDA روی)

سلول ۲۸: بنچمارک استنتاج مدل پایه و مدل کامپایل شده: در این سلول، یک تابع benchmark_inference نوشته شد که throughput را به صورت samples/sec محاسبه می‌کند. برای اینکه اندازه‌گیری دقیق‌تر باشد، ابتدا چند batch برای warmup اجرا شد تا اثر آماده‌سازی اولیه GPU و بارگذاری کرنل‌ها در زمان اصلی محاسبه تاثیر نگذارد. سپس روی تعداد مشخصی batch، زمان کل اندازه‌گیری شد و تعداد نمونه پردازش شده بر زمان تقسیم شد. در بنچمارک از torch.inference_mode برای حذف محاسبات گرادیان و از AUTOCast برای mixed precision استفاده شد. همچنین روی GPU از memory_format=channels_last استفاده شد تا دسترسی حافظه بهتر شود و سرعت بالاتری به دست آید.

نتایج بنچمارک نشان داد:

حالت اجرا	Samples	Seconds	Samples/sec
Base (model_final)	۳۵۲۰	۸.۱۰۴۳	۴۳۴.۳۴
Compiled (torch.compile)	۳۵۲۰	۸.۱۰۷۰	۴۳۴.۱۹

ضریب سرعت نیز برابر ۰.۹۹۹۷ گزارش شد، یعنی در این اجرا torch.compile باعث افزایش سرعت محسوس نشده و عملکرد تقریباً برابر با حالت پایه بوده است. با توجه به این خروجی، در این محیط خاص، یا گلوگاه اصلی سرعت مربوط به بخش‌هایی خارج از مدل بوده یا بهینه‌سازی torch.compile برای این معماری و این تنظیمات، سود مشخصی ایجاد نکرده است.

سلول ۲۹: بررسی صحت خروجی مدل پایه و مدل کامپایل شده: از آنجا که در بهینه‌سازی‌های سرعت ممکن است خروجی مدل به صورت عددی کمی تغییر کند، در این سلول خروجی دو مدل با یکدیگر مقایسه شد. مقایسه روی ۵ batch انجام شد و اختلاف عددی logits و همچنین تطابق پیش‌بینی top-۱ بررسی شد. برای پایدارتر بودن مقایسه، autocast در این مرحله غیرفعال شد تا logits در float۳۲ محاسبه شوند.

خروجی این سلول نشان داد:

مقدار	مورد
۵	batch بررسی شده
۶.۵۶e-۰۶	Max abs diff (logits)
۳.۷۹e-۰۷	Mean abs diff (logits)
۱.۰	Top-۱ prediction match
PASS	نتیجه

این نتایج نشان می‌دهد اختلاف عددی logits بسیار ناچیز است و پیش‌بینی نهایی مدل در همه نمونه‌های بررسی شده دقیقاً یکسان بوده است. بنابراین، از نظر صحت خروجی، کامپایل مدل تغییری در تصمیم طبقه‌بندی ایجاد نکرده است.

جمع‌بندی سوال ۸

در این بخش، ابتدا برای داشتن مقایسه منصفانه سرعت، یک DataLoader با batch ثابت ساخته شد. سپس سرعت استنتاج مدل پایه با سرعت مدل کامپایل شده توسط torch.compile اندازه‌گیری و مقایسه شد. نتیجه بنچمارک نشان داد در این اجرا سرعت تقریباً ثابت مانده و افزایش سرعت مشاهده نشده است. با این حال، بررسی صحت خروجی نشان داد خروجی مدل کامپایل شده از نظر عددی اختلاف بسیار ناچیزی با مدل پایه دارد و پیش‌بینی‌های نهایی کاملاً یکسان هستند؛ بنابراین اجرای torch.compile از نظر صحت خروجی قابل اعتماد است، هرچند در این محیط خاص بهبود سرعت ایجاد نکرد.

جمع بندی و نتیجه گیری

در مجموع، مدل پیاده‌سازی شده توانسته فرآیند تشخیص کلاس را انجام دهد و هدف پروژه را از نظر ساخت ViT از ابتدا، آموزش از صفر، ارزیابی و تحلیل خطاها برآورده کند. با این حال، نتایج نشان می‌دهد برای رسیدن به دقت بالاتر، نیاز به بهبود در داده، روش‌های منظم‌سازی و به‌ویژه بهره‌گیری از اطلاعات اولیه بهتر (مانند pretrained) یا طراحی ورودی‌های دقیق‌تر برای تمرکز بر نواحی کلیدی تصویر وجود دارد.